

# Phase 3: Feature Engineering Plan

## 1. Goal & Requirements

The goal of Phase 3 is to take the raw text of the candidate pairs generated in Phase 2 and transform them into numeric feature vectors that a machine learning model (like LightGBM or XGBoost) can understand.

For each candidate pair (S1\_Entity, S2/S3\_Entity), we must compute mathematical similarities between their respective business\_name, business\_address, zip\_pin, and blocking keys.

### 1.1 Requirements

- String Distance Metrics: Compute edit distances that account for typos.
- Token-based Metrics: Compute overlap of words to handle reordering (e.g., "Cafe Roma" vs "Roma Cafe").
- Phonetic Metrics: Check if words sound the same.
- Missing Value Handling: Robustly handle missing addresses or ZIP codes.
- High Performance: Feature extraction must process tens of millions of pairs quickly using optimized C/C++ backed libraries.

## 2. Feature Strategy

We will extract the following categories of features:

### 2.1 Name Similarity Features

| Feature Name         | Algorithm / Library        | Purpose                                            |
|----------------------|----------------------------|----------------------------------------------------|
| `name_jaro_winkler`  | `rapidfuzz.distance.Jar..` | Heavily weights matches at the beginning of the .. |
| `name_levenshtein..` | `rapidfuzz.distance.Lev..` | Captures character-level typos and substitutions.  |
| `name_token_jacca..` | Custom Set Intersection    | Handles out-of-order words (`len(A & B) / len(A    |
| `name_token_sort_..` | `rapidfuzz.fuzz.token_s..` | Handles completely scrambled words.                |
| `name_length_diff`   | `abs(len(A) - len(B))`     | Captures extreme abbreviations.                    |

### 2.2 Address Similarity Features

| Feature Name       | Algorithm / Library         | Purpose                                            |
|--------------------|-----------------------------|----------------------------------------------------|
| `addr_levenshte..` | `rapidfuzz.distance.Leve..` | Character-level typos in addresses.                |
| `addr_token_jac..` | Custom Set Intersection     | Very robust to missing tokens like "floor" or "s.. |
| `addr_cosine_sim`  | TF-IDF + Cosine             | Re-use our TF-IDF vectors from Phase 2 for a dir.. |

## 2.3 Exact & Heuristic Features

| Feature Name  | Logic                             | Purpose                                            |
|---------------|-----------------------------------|----------------------------------------------------|
| `zip_match`   | `1` if exact match, `0` if diff.. | Powerful geographic constraint.                    |
| `state_mat..` | Extract state from tokens and c.. | Broad geographic constraint.                       |
| `has_null_..` | Boolean                           | Lets the model know it shouldn't penalize `addr..` |

## 3. Potential Problems & Mitigation

### Problem 1: Computational Bottleneck

- The Issue: Computing Levenshtein distance natively in Python for 50 million pairs will take hours.
- The Solution: We must use rapidfuzz, which provides C++ optimized string distances. We will also use pandas vectorized operations (apply is often too slow, so we will use list comprehensions or numpy.vectorize). We can also process in chunks using multiprocessing.

### Problem 2: Handling Missing Data

- The Issue: Many records lack addresses. If we compute string distance between an empty string and a valid address, it results in 0 similarity. The ML model might incorrectly learn that "0 address similarity means it's not a match," even though it could be a perfect name match!
- The Solution: Instead of leaving similarity as 0, we will impute missing similarities with -1 (or np.nan since LightGBM handles NaNs natively). We also include explicit binary flags like is\_address\_missing = True.

### Problem 3: Memory Exhaustion

- The Issue: Expanding 50 million candidate pairs into a DataFrame with 15 float64 features will consume over 10GB of RAM.
- The Solution:
- Cast similarity float scores to float32.
- Cast boolean flags to int8.
- Write features directly to disk (Parquet or TSV) in chunks of 5 million pairs.

## 4. Development Workflow

---

- Feature Engineering Script: Create features.py.
- Test on Sample: Run the feature extractor on the 20k validation candidate pairs we generate in Phase 2.
- Distribution Check: Plot histograms of name\_jaro\_winkler for True Matches vs False Matches. We want to see a clear separation (bimodal distribution).
- Scale Up: Apply the script to the full candidate\_pairs.tsv in memory-safe chunks.